

```
[1] # Lab 10: MNIST Compression using Autoencoder (PyTorch)

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# -----
# 1. Hyperparameters
# -----
batch_size = 256
learning_rate = 1e-3
num_epochs = 10
latent_dim = 32

# -----
```




9.ipynb ☆

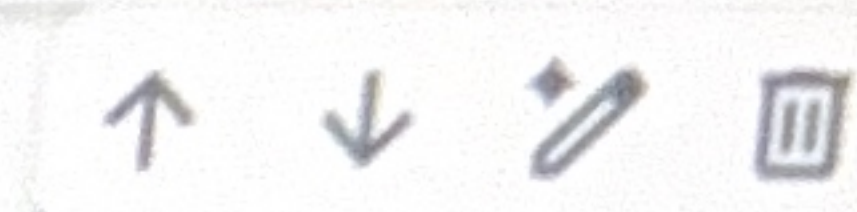
Edit View Insert Runtime Tools Help



Share

Commands + Code + Text ▶ Run all

Connect

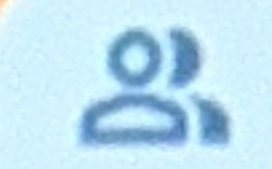


```
# -----  
# 2. MNIST Dataset  
# -----  
transform = transforms.Compose([transforms.ToTensor()])  
  
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)  
test_dataset = datasets.MNIST(root='./data', train=False, transform=transform, download=True)  
  
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)  
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)  
  
# -----  
# 3. Autoencoder Model  
# -----  
class Autoencoder(nn.Module):  
    def __init__(self):  
        super(Autoencoder, self).__init__()  
        # Encoder  
        self.encoder = nn.Sequential(  
            nn.Flatten(),
```





9.ipynb



Share



File Edit View Insert Runtime Tools Help

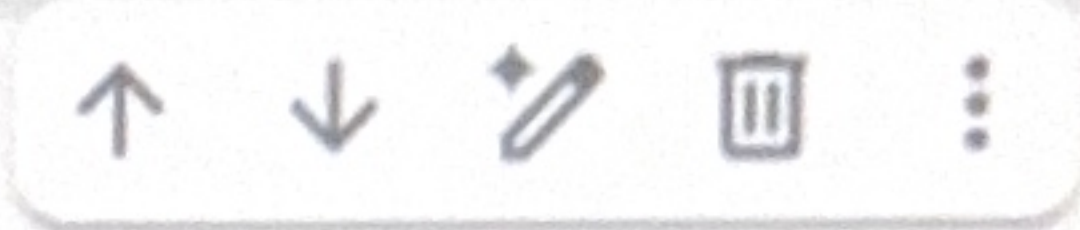
Commands + Code + Text Run all

Connect



```
nn.Linear(28*28, 128),
nn.ReLU(),
nn.Linear(128, 64),
nn.ReLU(),
nn.Linear(64, latent_dim)
)
# Decoder
self.decoder = nn.Sequential(
    nn.Linear(latent_dim, 64),
    nn.ReLU(),
    nn.Linear(64, 128),
    nn.ReLU(),
    nn.Linear(128, 28*28),
    nn.Sigmoid(),          # output between 0 and 1
    nn.Unflatten(1, (1,28,28))
)

def forward(self, x):
    x = self.encoder(x)
    x = self.decoder(x)
    return x
```



{ } Variables

Terminal





9.ipynb

File Edit View Insert Runtime Tools Help

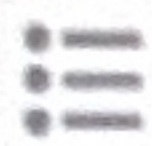


Share



Commands + Code + Text Run all

Connect



```
# 4. Model, Loss, Optimizer
# -----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Autoencoder().to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# -----
# 5. Training
# -----
losses = [] # To store average loss per epoch

for epoch in range(num_epochs):
    model.train()
    train_loss = 0
    for data, _ in train_loader:
        data = data.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, data)
        loss.backward()
```



Variables

Terminal




```
optimizer.step()
train_loss += loss.item() * data.size(0) # sum loss over batch

avg_loss = train_loss / len(train_loader.dataset)
losses.append(avg_loss)
print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.6f}")

# -----
# Plot Loss vs Epoch
# -----
plt.figure(figsize=(8,5))
plt.plot(range(1, num_epochs+1), losses, marker='o')
plt.title('Training Loss vs Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.grid(True)
plt.show()

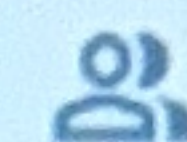
# -----
# 6. Testing & Visualization
# -----
```




9.ipynb



File Edit View Insert Runtime Tools Help

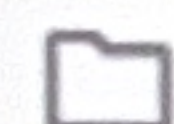
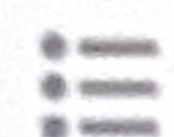


Share



Commands + Code + Text Run all

Connect



```
model.eval()
with torch.no_grad():
    for data, _ in test_loader:
        data = data.to(device)
        reconstructed = model(data)
        break # take first batch

# Display original and reconstructed images
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i+1)
    plt.imshow(data[i].cpu().squeeze(), cmap='gray')
    plt.title("Original")
    plt.axis('off')

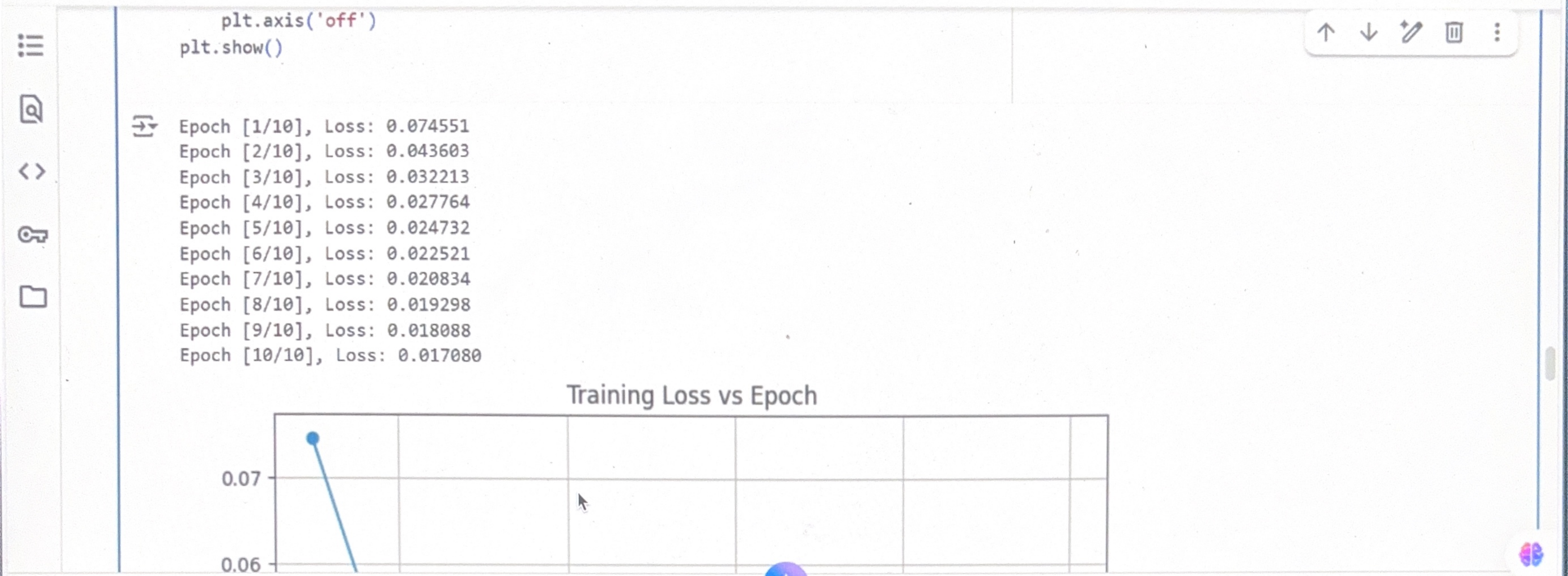
    # Reconstructed
    ax = plt.subplot(2, n, i+1+n)
    plt.imshow(reconstructed[i].cpu().squeeze(), cmap='gray')
    plt.title("Reconstructed")
```



{ } Variables

Terminal







Teams and... X Lab aim obj... X CIFAR-10 an... X cifar-10 dat... X CIFAR 10 da... X Google Driv... X Home - Goo... X 9.ipynb - Co... X

https://colab.research.google.com/drive/1n1Xghl5JqaDe3R2jFQdd5Sn9MUNRvhMf#scrollTo=e0qGdSqVtIbY

9.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

0.02

Epoch

2

4

6

8

10

0.015

0.01

0.005

0.001

Original

Original

Original

Original

Original

Original

Original

Original

Original

Original

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

Reconstructed

7

2

1

0

4

1

4

9

5

9

[]

import torch
import torch.nn as nn

Variables

Terminal

10. Perform Compression on MNIST dataset using autoencoder.

Aim:-

To compress and reconstruct the MNIST handwritten digit dataset using an autoencoder.

Objective:

1. To understand the concept of autoencoder for data compression & reconstruction.
2. To implement autoencoder using neural network framework.
3. To compress the MNIST dataset to lower dimensional representation.
4. To reconstruct the original images from the compressed representation.
5. To visualise compressed representation of the digit.

Pseudo Code:

1. Import required libraries.
2. Load MNIST dataset.
 - `x_train, x_test, y_train, y_test = mnist.load_data()`
 - Normalize data to range $[0, 1]$
3. Flatten image to vectors ($28 \times 28 = 784$)
4. Define the autoencoder architecture
 - Input layer = 784
 - Encoder layer = $128 \rightarrow 64 \rightarrow 32$
 - Decoder layer = $32 \rightarrow 64 \rightarrow 128 \rightarrow 784$
 - Activation function: ReLU for hidden layer, sigmoid for output layer.

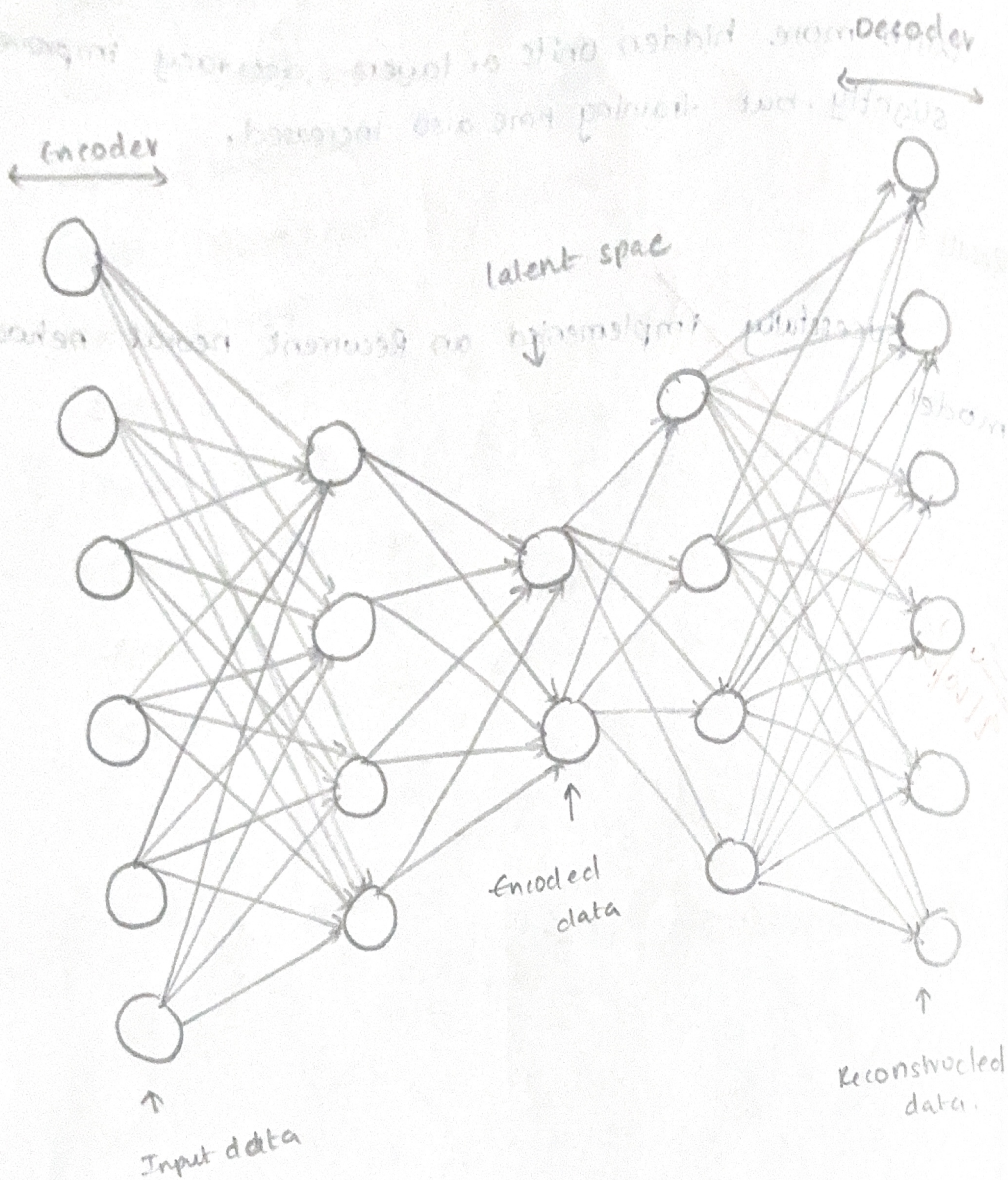
6. compile the model.
6. Train the autoencoder.
 - Input = x_{train} = output = x_{train}
7. Evaluate the model.
 - Reconstruct images from x_{test} .
8. visualize
 - original vs Reconstructed images.

Observation :-

1. The autoencoder successfully compressed MNIST images from 784 dimensions to 32 dimensions.
2. Reconstruct quality depends on the size of latent space & network depth.
3. Some fine details may be lost during compression.
4. Loss gradually decreases during training, showing that the network is reconstructing image effectively.

Result :-

Successfully implemented and performs compression on MNIST data using autoencoder.



Architecture of encoder

epoch (10/10)

epoch [1/10], loss : 0.074551

epoch [2/10], loss : 0.043603

epoch [3/10], loss : 0.032213

epoch [4/10], loss : 0.027764

epoch [5/10], loss : 0.024732

epoch [6/10], loss : 0.025252

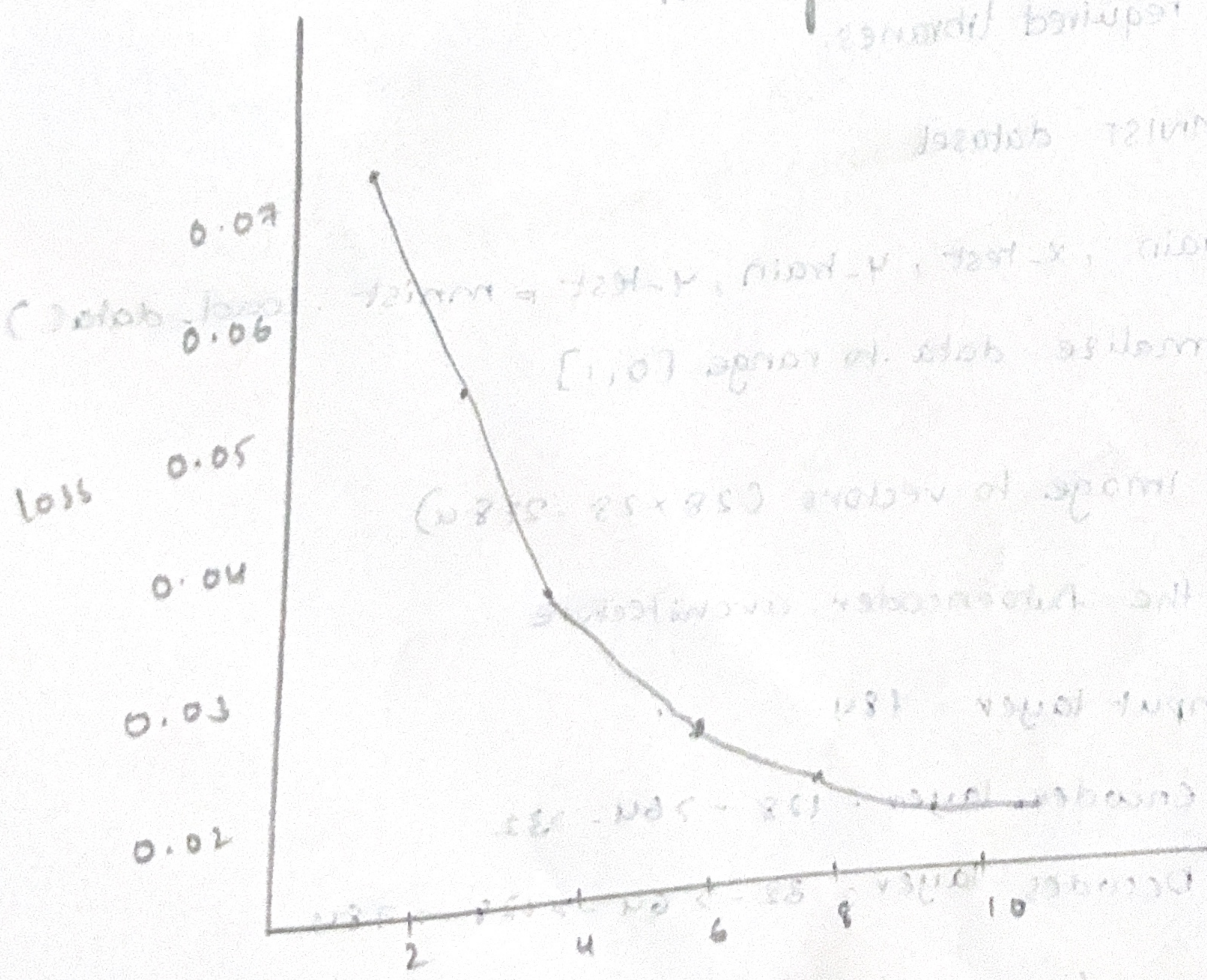
epoch [7/10], loss : 0.020634

epoch [8/10], loss : 0.019298

epoch [9/10], loss : 0.015088

epoch [10/10], loss : 0.017000

Training loss vs epoch.



epoch